

Question-1 Answer :

Assignment explanation :

Line-by-Line →

. Header File

```
#include<stdio.h>
```

Used for input and output functions like printf() and scanf().

. Main Function

```
int main()
```

Starting point of the program.

. 2D Array Declaration

```
int input[5][7]→Stores temperatures of: 5 farms,7 days
```

rows represent farms and columns represent days.

. Taking Input

```
for(int i = 0; i < 5; i++)
```

Outer loop for farms.

```
for(int j = 0; j < 7; j++)
```

Inner loop for days.

. Input Validation

```
while(1)→Runs until valid input is entered.
```

```
if(input[i][j] >= 5 && input[i][j] <= 60)→Checks whether temperature is between 5 and 60.
```

```
break;→Stops loop if valid input.
```

. Farm-wise Summary

```
int sum = 0;→stores total temperature of one farm.
```

```
sum = sum + input[i][j];→adds temperatures.
```

. Heat Stress Days

if(input[i][j] > 40)→counts days above 40°C.

. Cold Stress Days

if(input[i][j] < 10)→counts days below 10°C

. Average Calculation

(Average Temperature = $\frac{\text{Sum of 7 days temperatures}}{7}$)

float avg = sum / 7.0;→calculates average temperature.

. Condition Classification

if(avg >= 30)→High Heat Stress.

else if(avg >= 20 && avg < 30)→Optimal condition.

Else→Low Temperature Stress.

. Highest and Lowest Temperature

int highest = input[0][0];

int lowest = input[0][0];→initial the highest and lowest values

if(input[i][j] > highest)→finds highest temperature.

if(input[i][j] < lowest)→finds lowest temperature.

.Daily Average Temperature

for(int j = 0; j < 7; j++)→loop for each day.

sum = sum + input[i][j];→adds temperatures of all farms for one day.

. Daily Average Formula

(Daily Average = $\frac{\text{Sum of temperatures of all farms}}{5}$)

float daily_avg = sum / 5.0;→calculates average for each day.

. Hottest Day

if(daily_avg > hottest_avg)→finds highest average day.

. Coldest Day

if(daily_avg < coldest_avg)→finds lowest average day.

Question-2 Answer:

Most Interesting Problem Solved in SPL Course →

One of the most interesting problems I solved during my SPL course was printing a “Half Pyramid Pattern” using the C programming language. At first, the problem looked very simple, but while solving it, I learned many important programming concepts, especially about nested loops and pattern generation. I enjoyed this problem because it helped me understand how loops work together to create shapes and patterns. It also improved my logical thinking and problem-solving skills. When I first saw the pyramid output, I was confused about how stars increase row by row, but after practicing, I understood the logic clearly.

Problem Description : The program prints a half pyramid using star (*) symbols.

Example output for 5 rows:

```
*  
* *  
* * *  
* * * *  
* * * * *
```

Coding Logic

Step 1: Taking Input

First, I took the number of rows from the user.

```
int n;  
scanf("%d", &n);
```

Here, n represents how many rows the pyramid will have.

Step 2: Using the Outer Loop

The outer loop controls the rows of the pyramid.

```
for(int i = 1; i <= n; i++)
```

In this loop:

- i represents the current row number.
- The loop starts from 1 and continues until n.

For example:

- When i = 1, the first row is printed.
- When i = 2, the second row is printed.

Step 3: Using the Inner Loop

The inner loop is responsible for printing stars in each row.

```
for(int j = 1; j <= i; j++)
```

Here:

- j controls the number of stars.
- The number of stars depends on the current row number (i).

Example:

- Row 1 → 1 star
- Row 2 → 2 stars
- Row 3 → 3 stars

This is how the pyramid shape is created.

Step 4: Printing Stars

```
printf("* ");
```

This statement prints one star followed by a space.

Step 5: Moving to the Next Line

```
printf("\n");
```

After printing all stars of one row, this statement moves the cursor to the next line so the next row can be printed properly.

C Code→

```
#include<stdio.h>
```

```
int main()
```

```

{
    int n;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    for(int i = 1; i <= n; i++)
    {
        for(int j = 1; j <= i; j++)
        {
            printf("* ");
        }

        printf("\n");
    }

    return 0;
}

```

What I Learned

From solving this problem, I learned many important concepts of C programming, such as:

- a.How nested loops work
- b.How to create patterns using loops
- c.How outer and inner loops interact with each other
- d.How to think logically while solving programming problems

This problem made programming more interesting and enjoyable for me. It also increased my confidence in working with loops and pattern-related problems in C.